

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

FACULTAD DE CIENCIAS PURAS — CARRERA DE INGENIERÍA INFORMÁTICA

INF781 · SEGURIDAD DE SOFTWARE

EXAMEN FINAL

Proyecto integrador: Billetera Electrónica Segura — “SecureWallet”

Docente:	M. Sc. Huáscar Fedor Gonzales Guzmán	Gestión:	I/2026
Modalidad:	Proyecto en equipo (2–3 estudiantes)	Ponderación:	100 puntos
Entrega:	Repositorio GitHub + defensa oral		

1. COMPETENCIA A EVALUAR

Diseña, implementa y verifica una aplicación completa (frontend + backend API REST) aplicando de forma sistemática controles de seguridad contra las vulnerabilidades del **OWASP Top 10 (2021)**, gestionando autenticación robusta, autorización granular, protección de datos sensibles y trazabilidad de operaciones financieras, con actitud crítica y ética profesional.

2. DESCRIPCIÓN DEL PROYECTO

Deberá desarrollar **SecureWallet**, una billetera electrónica que permite a los usuarios registrar una cuenta, cargar saldo, enviar y recibir dinero entre usuarios de la plataforma, y consultar su historial de movimientos. Al tratarse de una aplicación financiera, la seguridad **no es una característica opcional: es el objeto central de la evaluación**. Un sistema funcional pero inseguro será calificado como insuficiente.

2.1. Arquitectura obligatoria

El sistema se compone de dos piezas desacopladas que se comunican exclusivamente por HTTP/JSON:

COMPONENTE	OPCIÓN A	OPCIÓN B
Backend (API REST)	Laravel 13 (PHP ≥ 8.3) + Sanctum o JWT	NestJS (Node ≥ 20) + Passport-JWT
Frontend	Aplicación web SPA (React, Vue o Angular)	Aplicación móvil (Flutter, React Native o Kotlin/Swift)
Base de datos	PostgreSQL o MySQL	PostgreSQL o MySQL

NOTA

COMPONENTE	OPCIÓN A	OPCIÓN B
Puede combinar libremente: por ejemplo, backend NestJS con frontend web, o backend Laravel 13 con app móvil. Lo obligatorio es que exista un frontend (web o móvil) consumiendo un backend API REST propio. No se admiten monolitos con vistas de servidor (Blade/EJS) como frontend.		

3. REQUERIMIENTOS FUNCIONALES (30 PTS)

CÓDIGO	REQUERIMIENTO
RF-01	Registro de usuario con nombre completo, CI, correo, teléfono y contraseña. Al registrarse se crea automáticamente una billetera con saldo 0.00 Bs.
RF-02	Inicio de sesión con correo y contraseña. Tras 5 intentos fallidos consecutivos la cuenta se bloquea temporalmente por 15 minutos.
RF-03	Verificación en dos pasos (MFA/TOTP) con Google Authenticator: activación mediante código QR y exigencia del código de 6 dígitos en el login cuando esté habilitada.
RF-04	Consulta de saldo y datos del perfil del usuario autenticado.
RF-05	Recarga de saldo (simulada): el usuario ingresa un monto y su saldo se incrementa. Registrar la operación como movimiento tipo RECARGA.
RF-06	Transferencia entre usuarios: envío de dinero a otro usuario identificado por correo o número de teléfono, con verificación de saldo suficiente, monto mínimo 1.00 Bs y máximo 5 000.00 Bs por operación.
RF-07	Confirmación de transferencia en dos pasos: antes de ejecutarse, el sistema muestra el nombre del destinatario y solicita confirmación explícita (y el código TOTP si el monto supera 500.00 Bs).
RF-08	Historial de movimientos paginado del usuario autenticado (recargas, envíos y recepciones), con fecha, tipo, monto, contraparte y saldo resultante.
RF-09	Panel de administrador: listado de usuarios, bloqueo/desbloqueo de cuentas y consulta de la bitácora de auditoría. Solo accesible al rol ADMIN.
RF-10	Cierre de sesión que invalide efectivamente el token/refresh token en el servidor (no solo borrado en el cliente).

4. REQUERIMIENTOS DE SEGURIDAD (40 PTS)

Cada control se evaluará contra la categoría OWASP indicada. Durante la defensa, el docente ejecutará **pruebas de intrusión básicas** (peticiones manipuladas con Postman/curl) para verificar que los controles realmente funcionan.

CÓD.	OWASP	CONTROL EXIGIDO
RS-01	A01 – Broken Access Control	Autorización a nivel de objeto (BOLA/IDOR): un usuario jamás puede consultar el saldo, movimientos ni perfil de otro usuario manipulando IDs en la URL o el body. Usar UUID como identificador público de billeteras y transacciones.
RS-02	A01 – Broken Access Control	RBAC con al menos dos roles (USER, ADMIN). Los endpoints administrativos deben rechazar con 403 a usuarios sin rol ADMIN, verificado en el servidor (guard/middleware/policy), nunca solo ocultando botones en el frontend.

CÓD.	OWASP	CONTROL EXIGIDO
RS-03	A02 – Cryptographic Failures	Contraseñas con bcrypt (cost ≥ 12) o argon2id. Prohibido MD5/SHA1 o texto plano. El secreto TOTP y los datos sensibles no deben exponerse nunca en respuestas JSON.
RS-04	A03 – Injection	Toda consulta a BD mediante ORM/Query Builder con parámetros enlazados (Eloquent/TypeORM/Prisma). Validación estricta de entrada en todos los endpoints (Form Requests en Laravel / class-validator + ValidationPipe whitelist en NestJS): tipos, rangos, formatos y rechazo de campos no esperados (mass assignment).
RS-05	A04 – Insecure Design	Integridad transaccional: la transferencia debe ejecutarse dentro de una transacción de BD con bloqueo (SELECT ... FOR UPDATE o equivalente) que impida condiciones de carrera y saldos negativos ante peticiones concurrentes. Implementar clave de idempotencia (header Idempotency-Key) para evitar dobles cobros por reintentos.
RS-06	A05 – Security Misconfiguration	CORS restringido al origen del frontend; cabeceras de seguridad (helmet en NestJS / middleware equivalente en Laravel); mensajes de error genéricos en producción (sin stack traces ni SQL); variables sensibles solo en .env (nunca en el repositorio).
RS-07	A07 – Identification & Authentication Failures	Autenticación con access token de vida corta (≤ 15 min) y refresh token persistente con rotación y detección de reutilización (revocar toda la familia ante un refresh token ya usado). Política de contraseñas: mínimo 10 caracteres, mayúscula, minúscula, dígito y símbolo.
RS-08	A07 – Identification & Authentication Failures	Rate limiting: máximo 5 intentos/minuto en login y 10 peticiones/minuto en transferencias, con respuesta 429. CAPTCHA (reCAPTCHA/hCaptcha o propio) en el registro.
RS-09	A09 – Security Logging & Monitoring	Bitácora de auditoría inmutable: registrar login exitoso/fallido, activación de MFA, bloqueos de cuenta, recargas y transferencias, con usuario, IP, user-agent y timestamp. Consultable solo por ADMIN.
RS-10	XSS / CSRF (transversal)	Salida escapada en el frontend (sin innerHTML/v-html sin sanitizar); si usa cookies para tokens: HttpOnly + Secure + SameSite. Si usa Authorization: Bearer, justificar el almacenamiento elegido en el cliente y sus riesgos.

5. ENDPOINTS MÍNIMOS DE LA API

Prefijo base: `/api/v1`. Acceso: **Público** (sin token), **USER** (token válido) o **ADMIN** (token + rol).

MÉTODO	ENDPOINT	ACCESO	DESCRIPCIÓN
POST	<code>/auth/register</code>	Público	Registro + CAPTCHA. Crea usuario y billetera.
POST	<code>/auth/login</code>	Público	Login. Si MFA activo, exige paso TOTP.
POST	<code>/auth/mfa/verify</code>	Público*	Verifica código TOTP y emite tokens.
POST	<code>/auth/refresh</code>	Público*	Rota el refresh token (detección de reuso).
POST	<code>/auth/logout</code>	USER	Revoca tokens del usuario en servidor.

MÉTODO	ENDPOINT	ACCESO	DESCRIPCIÓN
POST	/auth/mfa/enable	USER	Genera secreto TOTP + QR de activación.
GET	/me	USER	Perfil del usuario autenticado.
GET	/wallet	USER	Saldo actual de la billetera propia.
POST	/wallet/topup	USER	Recarga de saldo (simulada).
POST	/transfers	USER	Crea transferencia (Idempotency-Key obligatorio).
POST	/transfers/{uuid}/confirm	USER	Confirma la transferencia pendiente (+TOTP si > 500 Bs).
GET	/transactions	USER	Historial propio, paginado y filtrable por tipo.
GET	/admin/users	ADMIN	Listado de usuarios.
PATCH	/admin/users/{uuid}/block	ADMIN	Bloquea o desbloquea una cuenta.
GET	/admin/audit-logs	ADMIN	Bitácora de auditoría, paginada.

* Endpoints públicos pero con credencial intermedia (ticket MFA o refresh token válido). Puede añadir endpoints adicionales si su diseño lo requiere, documentándolos.

5.1. Ejemplo de contrato — POST /api/v1/transfers

```
POST /api/v1/transfers
Authorization: Bearer <access_token>
Idempotency-Key: 9f1c2a7e-55b4-4d2c-a1f0-3c8e7d6b1a90

{
  "destinatario": "70123456",           // correo o teléfono
  "monto": 750.50,
  "descripcion": "Pago almuerzo"
}

→ 201 Created
{
  "uuid": "b3d9...", "estado": "PENDIENTE_CONFIRMACION",
  "requiere_totp": true, "expira_en": 120
}
```

6. ENTREGABLES

E1	Repositorio GitHub con dos carpetas: /backend y /frontend.
E2	README.md con: instrucciones de instalación y ejecución, variables de entorno de ejemplo (.env.example), usuarios semilla (1 ADMIN y 2 USER con MFA de prueba) y credenciales de demo.
E3	Colección Postman/Insomnia exportada con todos los endpoints y ejemplos de peticiones válidas e inválidas.

7. DEFENSA ORAL

La defensa es individual dentro del equipo. Cada integrante debe ser capaz de explicar y modificar en vivo cualquier parte del código, incluida la implementada por sus compañeros. Durante la defensa el docente: (1) ejecutará el flujo funcional completo, (2) lanzará ataques básicos con Postman/curl contra los controles RS-01 a RS-10, y (3) solicitará cambios menores en vivo (p. ej., ajustar el límite de rate limiting o el costo de bcrypt) para verificar autoría. **El uso de asistentes de IA está permitido durante el desarrollo, pero no durante la defensa;** no saber explicar el propio código se considera evidencia de no autoría y anula la nota individual.

8. RÚBRICA DE EVALUACIÓN (100 PTS)

N°	CRITERIO	DESCRIPCIÓN	PTS
1	Requerimientos funcionales (RF-01 a RF-10)	Flujo completo operativo de registro, autenticación, recarga, transferencia con confirmación, historial y panel admin.	30
2	Controles de seguridad (RS-01 a RS-10)	4 pts por control verificado en vivo. Controles parcialmente implementados: 2 pts. Sujeto a vulnerabilidades eliminatorias (Sección 4).	40
3	Calidad técnica	Arquitectura por capas, código limpio, migraciones y seeders, manejo consistente de errores JSON, uso correcto de códigos HTTP.	10
4	Entregables y documentación	Repositorio, README reproducible y colección Postman completa.	10
5	Defensa individual	Dominio del código propio y del equipo, respuesta a preguntas conceptuales OWASP, modificaciones en vivo.	10
TOTAL			100

9. CHECKLIST DE VERIFICACIÓN PREVIA A LA ENTREGA

✓	El proyecto levanta desde cero siguiendo únicamente el README (migraciones + seeders incluidos).
✓	Ningún endpoint expone hash de contraseña, secreto TOTP ni refresh tokens en el JSON.
✓	Manipular el UUID de otra billetera/transacción devuelve 403/404, nunca datos ajenos.
✓	Dos peticiones concurrentes de transferencia no generan saldo negativo ni doble débito.
✓	Reutilizar un refresh token ya rotado revoca toda la sesión (familia de tokens).
✓	El 6.º intento de login en menos de un minuto responde 429 y bloquea tras 5 fallos.
✓	El archivo .env no está en el repositorio; existe .env.example.
✓	La bitácora registra IP y user-agent de cada login y transferencia.
✓	El frontend nunca decide la autorización: todo control se repite en el servidor.



El video demuestra el flujo completo, incluido un ataque rechazado.

ÉTICA PROFESIONAL

Este examen simula un producto financiero real. Trate los datos de prueba con la misma seriedad con la que trataría datos reales: la seguridad de software es, ante todo, una responsabilidad con las personas que confían en nuestros sistemas.